Benchmarking Supervised Machine Learning Algorithms using PyCaret for Binary and Multiclassification of Network Anomaly Attacks

Manish Katel, Kiran Malla and Nick Rahimi
The University of Southern Mississippi, Hattiesburg MS 39406, USA
manish.katel@usm.edu, kiran.malla@usm.edu, nick.rahimi@usm.edu

***Abstract*—Machine learning models for intrusion detection vary significantly in terms of performance and training time. Model performance depends on data preparation; however, even when trained on same data preparation pipeline, different models produce different results. Some models achieve higher performance but require a greater computational cost, some models show high detection accuracy but show biasness towards certain attacks making model selection vital. To address these challenges, this study presents a comprehensive, memory-efficient benchmarking framework for the CICIDS 2017 dataset, integrating systematic preprocessing, feature reduction, data balancing, and an AutoML-based model comparison to identify the optimal supervised learning algorithms for network anomaly detection. The PyCaret AutoML library enabled simultaneous training and comparison of upto 16 supervised algorithms across multiple evaluation metrics, including accuracy, recall, precision, F1 score, and training time. Experimental findings reveal that XGBoost achieves optimal performance for binary classification (99.21% accuracy, 0.158s training time), while Extra Trees Classifier excels in multiclass scenarios (99.40% accuracy, 0.670s training time). Furthermore, confusion matrix analysis identifies attack-specific model strengths: LGBM achieves highest per-attack accuracy (recall) for benign traffic, CatBoost maximizes Bot detection accuracy, and Random Forest performs best for Brute Force and Port Scan attacks. This work demonstrates that combining memory-efficient preprocessing with automated benchmarking pipelines enable comprehensive model comparison, providing practitioners with proper guidelines for selecting algorithms based on accuracy, computational efficiency, and specific threat detection requirements.**

***Index Terms*—PyCaret, Intrusion Detection, SMOTE, CICIDS-2017, AutoML, Supervised Learning**

I. Introduction

Modern network environments have become increasingly vulnerable as cyberattacks evolve faster than traditional security tools can adapt. Signature-based antivirus systems are very effective against known malware, but they fail to detect new threats because they depend on predefined signature attacks [1], [2]. To address these limitations, machine learning-based intrusion detection systems (IDS) have become essential for monitoring real-time network behavior and identifying deviations from normal traffic patterns [2]. However, anomaly-based IDS still face challenges such as high-dimensional feature spaces, rapidly changing traffic distributions, severe class imbalance, memory-heavy technique usage, which often lead to unstable model performance and slow attack detection which clearly shows us that supervised learning with proper memory-saving techniques are necessary.

Supervised machine learning has become a dominant approach for anomaly detection due to its ability to extract discriminative patterns from labeled network data [3]. To gain strong performance from supervised learning models, careful preprocessing of data is necessary. However, strong performance is still not guaranteed in practice even if processes like missing-value handling, duplicate value removal, outlier detection, feature reduction, and dataset balancing play a decisive role in determining model effectiveness. Suitable model selection according to the dataset is also an important step. Without careful attention to these steps, even advanced classifiers can yield very high or low evaluation results, which are both bad. This is where AutoML, especially PyCaret, comes into play.

Automated Machine Learning (AutoML) has evolved rapidly over the past decade to reduce the expertise, time, and effort required to design high-performance machine learning models. Initially, AutoML research focused on hyperparameter optimization and algorithm selection, but now it is advancing towards automating entire machine learning pipelines [4].

PyCaret is a practical, low-code AutoML library which helps to perform most common machine learning tasks with minimal code including data preprocessing, model training, comparison, and evaluation [5]. PyCaret integrates with popular libraries such as scikit-learn, XGBoost, LightGBM, and CatBoost and helps us to compare models based on multipe evaluation metrics even upto 28 different machine learning models for classification according to the dataset which is the main reason it was used for this research [5], [16]. The general workflow of PyCaret can be seen in Fig. 1.

The rest of the paper is organized as follows. Section II includes the general literature review of network anomaly detection and then the background of AutoML and PyCaret. All parts of the proposed solution and the underlying technologies are described in Section III. This section is divided into 7 subsections, each discussing a distinct segment of the proposed solution: Dataset Overview, Data Grouping and Attacks Ingestion, Data Cleaning, Data Preprocessing, Feature Selection and Dimensionality Reduction, Data Splitting and Sampling, and, finally, PyCaret Model Setup and Comparison. The PyCaret comparison results, aggregated models ranking

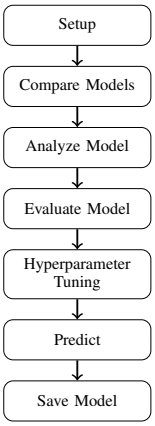


Fig. 1: General PyCaret Pipeline

based on higher accuracy and time taken and models ranking based on performance on individual attacks as discussed in the IV Section. Section V contains the comparison of our work with other works done previously in the field for the validity test on how good our results actually are. Then come the future considerations from our work on Section VI and discussion of the experiment and outputs on Section VII. Finally, at the end of the paper, some concluding remarks are given.

II. Literature Review

Recent studies using the CICIDS 2017 dataset highlight the importance of strong preprocessing pipelines and balanced sampling strategies for reliable intrusion detection. Zhen Yang et al. presented a systematic literature review of different methods like data preprocessing, attack-detection techniques and evaluation metrics on CICIDS 2017 dataset and other datasets for network anomaly detection [13]. The paper concluded that the nature of these network anomaly datasets requires a solid foundation of data preprocessing techniques, model selection and evaluation metrics because of high dimensionality and complexity of the data. They surveyed and presented the most common data preprocessing techniques, most common models, most common evaluation metrics and most common datasets used for machine learning based network anomaly detection in recent years, but it still lifts the doubt that what are the actual most effective techniques even among those common techniques to build a solid high performing baseline model. The actual most techniques are not only those which just provide good accuracy, but those that are written with minimal code and use memory saving techniques providing low latency on real time anomaly attack detection.

Ashim Dahal et al. presented with the analysis of zero-day attack detection using modern deep learning technique Multi-Layer Perceptron (MLP) for attack detection and Explainable AI (XAI) for model explanation on KDD-99 Cup dataset [11], [12]. The dataset was published in the year 1999 and contains four types of anomaly attacks which are very less for the current scenario as hackers have developed many types of attacks since then. They applied modern deep learning techniques for model building, but the model suffered from overfitting and MLP is very computationally extensive as well. Therefore, it was important to choose the dataset that has more attack variations and to build a computationally efficient and accurate anomaly attack detection model for which CICIDS 2017 dataset found to be perfect. If data preprocessing techniques and models are carefully chosen, then we can certainly lay down the models that can accurately detect anomaly attacks.

Even beyond MLP, a lot of machine learning and deep learning techniques have been developed in recent years which makes us think about what method to choose. Traditional ML definitely has flaws for which deep learning methods were developed, but deep learning methods also require heavy processing power. It takes a lot of time to develop a model. We must preprocess the dataset and spend a lot of time on model training and hyperparameter optimization. Therefore, AutoML was introduced so that we can automate machine learning tasks. It saves our time as it automatically prepares data for model training in a few lines of code and gives us the comparison of different supervised learning models for the dataset on multiple evaluation metrics.

However, only few intrusion detection studies have applied AutoML directly to CICIDS-2017 dataset. And there's a reason for that. Haowen Liu et al. suggested that AutoML systems rely on large-scale ensembling due to which it faces challenges in real-time inference due to latency and memory requirements. Their comparison of different approaches for building models for network intrusion detection also concluded that modern and advanced transformer-based models may scale well in terms of representation learning but demand extensive GPU support [9]. Stephanie Ness et al. paper compared and revealed that using supervised learning models like XGBoost and LightGBM alone exhibit achieve perfect training accuracy of 1.0 others like Isolation Forest show limitations with low accuracy in KDD Cup 99 dataset [10]. So, we can say that supervised learning alone might also be infeasible for network anomaly detection. Therefore, there comes the need to smoothen supervised learning - a better preprocessing technique selection, better model selection, and evaluation metrics selection as well.

III. Materials and Methods

This study employed a systematic machine-learning pipeline consisting of data integration, cleaning, preprocessing, dimensionality reduction, resampling, automated model comparison, and evaluation.

All experiments were performed in virtual environment made using Python 3.9 and PyCaret 3.1 on Jupyter Notebook as the development environment. Virtual environment was created because PyCaret automatically installs the version of numpy, pandas, sklearn and other machine learning libraries which can clash with those libraries installed on the system. The CPU consisted of an AMD Ryzen 7 5825 U processor and 16 GB of RAM which provided sufficient resources for handling the dataset and building the model. PyCaret compares

16 supervised algorithms at once so it might take a bit more time depending on the device and can cause device heating due to a lot of power consumption. Finally, Training Time (TT sec) is a hardware-dependent metric, meaning results may vary across different computational environments.

All experiments were performed on the CICIDS-2017 dataset for both binary and multiclassification. The complete machine learning pipeline workflow is illustrated in Fig. 2.

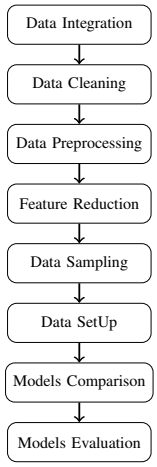


Fig. 2: Complete Machine Learning Pipeline for the Experiment

A. Dataset Overview

The CICIDS-2017 dataset, developed by the Canadian Institute for Cybersecurity, is one of the most realistic and widely used benchmark dataset for intrusion detection research because includes modern cyberattacks and normal network traffic in a setup that looks like a real enterprise network [3]. The original CICIDS 2017 dataset is available in Kaggle for free from where we used the dataset for our experiment (https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset). The dataset consists of total 22,362,897 entries and 79 features. These features comprise a range of network metrics like the total length of Fwd. Packets, Average Packet Size, Flow Duration, Total Fwd Packets, Total Backward Packets, Total Length of Fwd. Packets, Label and so on. The "Label" consists of attack types where "Benign" is considered as normal attack type and other attack categories such as DoS/DDoS, Brute Force, Web attacks, Botnet activity, and Infiltration as anomaly attacks. Huge class imbalance of 80.30% of the attacks as benign and rest of 19.70 % of attacks as anomaly attacks was found after data analysis. Originally there were 15 different types of attacks, but similar attacks were grouped together to make it 9 which is discussed in Section III-B. Two different datasets were sampled and used which are discussed in Section III-E.

B. Data Grouping and Attacks Integration

The key challenge with the CICIDS-2017 dataset was that it was distributed across eight independent CSV files, with feature names following inconsistent naming conventions and many attack labels exhibiting highly uneven class distributions. These issues complicate preprocessing and can bias supervised learning models toward dominant classes. To resolve this, all CSV files were first merged into a single unified data frame, and column names were standardized by converting them to lowercase to ensure consistency across experiments.

Another problem was the large number of attack labels. Many of them looked different in terms of name but behaved almost the same. Keeping them separate increased class imbalance and made the models focus on labels instead of real attack behavior. To fix this, semantically similar attacks such as Web Brute Force, Web XSS, and Web SQL Injection were consolidated into a single Web Attack class; DoS Hulk, DoS GoldenEye, DoS Slowloris, and DoS SlowHTTPTest were merged under DoS; and FTP Patator and SSH Patator were combined into a Brute Force category. The remaining attack types were kept same resulting in nine consolidated categories, including benign traffic as shown in Tables I and II.

TABLE I: Attack Types Before Grouping

Attack type	Frequency
BENIGN	2,096,484
DoS Hulk	172,549
DoS GoldenEye	10,286
DoS Slowloris	5,385
DoS Slowhttptest	5,228
DDoS	128,016
Port Scan	90,819
FTP-Patator	5,933
SSH-Patator	3,219
Brute Force	9,152
Web Attack – Brute Force	2,143
Bot	1,953
Infiltration	36
Heartbleed	11
Web Attack – SQL Injection	21
Web Attack - XSS	652

TABLE II: Attack Types After Grouping

Attack type	Frequency
BENIGN	2,096,484
DoS	193,748
DDoS	128,016
Port Scan	90,819
BruteForce	9,152
Web Attack	2,143
Bot	1,953
Infiltration	36
Heartbleed	11

C. Data Cleaning

A total of 308,381 duplicate records were identified and removed from the combined dataset as eliminating duplicates prevents biased learning and reduces unnecessary computation.

Some features contained infinite values which can't be processed by model. To clean the data, all infinite values were first converted to NaN, and the total number of missing values

- including both NaNs and converted infinities were calculated. Missing values were then imputed using the median as it is suitable for the skewed distributions. The details of infinite values and missing values can be seen in Table III.

TABLE III: Missing Values Count

Type	Count
Missing Values	353
Infinite Values	2,775
Total Missing Values	3,128

Outliers can distort anomaly detection and weaken relationships between features, which can reduce model performance. To address this, Interquartile Range (IQR) was used. Isolation Forest Model was also tested before to get more accurate separation of outliers, but as the result of Interquartile Range (IQR) method was similar to Isolation Forest method, IQR method was chosen because it took less time to compile and show the output while Isolation Forest model took longer time to train. Using this method, a total of 126,119 outliers were detected and removed because it improves the stability of the dataset and makes the models more reliable.

D. Data Preprocessing

The categorical attack-type labels were converted to integer codes (label encoding). Since these labels are nominal identifiers (with no natural order), label encoding provides a simple numeric representation required by classifiers without implying any ordinal relationship. This enables all supervised models to use the class labels directly.

All numerical features were scaled to have zero mean and unit variance (z-score normalization). This standardization ensured that each feature contributed equally to model training. Normalization also improves the effectiveness of PCA-based dimensionality reduction [8].

E. Feature Selection and Dimensionality Reduction

Features with zero variance or only a single unique value were removed as they provide no useful information for learning and can negatively affect model performance.

Due to the high dimensionality of the CICIDS-2017 dataset, IPCA was used instead of standard PCA because it is a more effective method for high dimensional data. This approach allowed the original 79-dimensional feature space to be reduced to 35 principal components while preserving 99.30% of the total variance. Reducing the dimensions improves the model's efficiency in real time as models need to process less data for prediction.

F. Data Splitting and Sampling

The dataset was then split into 70% training and 30% testing sets. The training set was then used to do the sampling and create the binary and multi classification datasets. Sampling was done after splitting to prevent the data leakage.

For the binary task (Benign vs. Attack), majority class down sampling was applied to create a balanced dataset. After down sampling, 10,034 samples of anomaly attacks and 9,936 samples of 'Benign' attacks were formed. Down sampling was used instead of oversampling because minority samples data are very critical, so it is better to keep the minority samples as much as they are and reduce the majority samples [7].

The multiclass dataset had extreme class imbalance, which can reduce model accuracy for minority classes. Therefore, SMOTE (Synthetic Minority Oversampling Technique) was applied instead of other oversampling techniques because it just not replicates minority samples but interpolates between neighboring minority-class samples creating a more uniform distribution and preserving the majority class data allowing the model to learn effectively across all attack categories [6]. Tables IV and V show the attack types before and after using SMOTE.

TABLE IV: Attack Types Before SMOTE

Attack type	Frequency
BENIGN	2,096,484
DoS	193,784
DDoS	128,016
Port Scan	90,819
Brute Force	9,152
Web Attack	2,143
Bot	1,953
Infiltration	36
Heartbleed	11

TABLE V: Attack Types After SMOTE

Attack type	Frequency
Benign	8,000
DoS	8,000
DDoS	8,000
Port Scan	8,000
BruteForce	8,000
Web Attack	8,000
Bot	8,000

G. PyCaret Model Setup and Comparison

The final stage of the methodology focused on supervised model training and performance comparison using the PyCaret framework. This automation made it easier to test and compare a wide range of supervised learning models. Previous studies have also shown that AutoML based approaches can simplify the development of intrusion-detection systems and support fair model comparison by reducing manual intervention and experimental variability [8], [9].

In our PyCaret setup, we explicitly disabled its internal preprocessing (preprocess=False) because all cleaning, normalization, and feature reduction had been done before manually. Then our training and testing datasets we had created were defined. 10-fold stratified cross-validation was also applied as it helps distribute data in training and testing set in a fair manner to prevent overfitting. Learning curves of top models for binary classification and multi-classification as classified by PyCaret can be seen in Figs. 3 and 4.

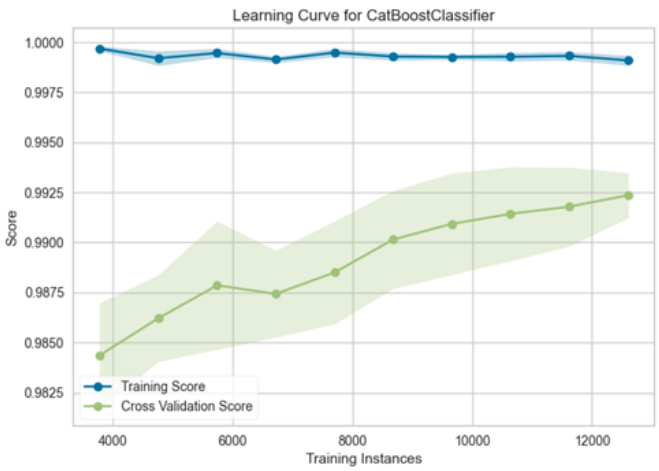


Fig. 3: (a) Learning Curve – Binary (CatBoost)

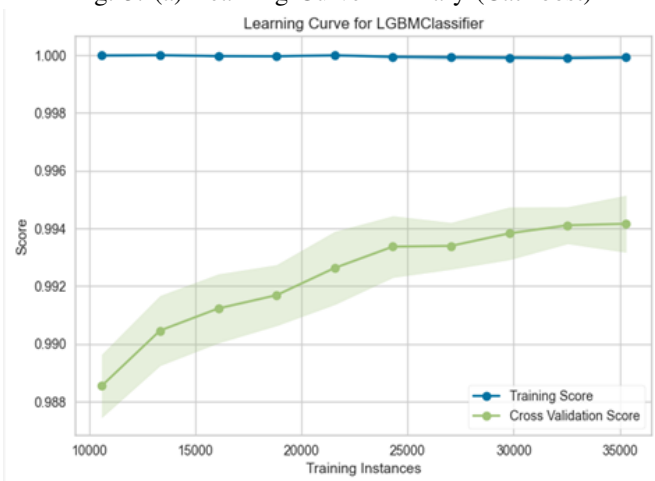


Fig. 4: (b) Learning Curve – Multi-class (LightGBM)

In Fig. 3 the training score remains stable at approximately 99.9% while the cross-validation score steadily improves from 98.4% to 99.2% as training instances increase and in Fig. 4 training accuracy remains consistent at 100% and cross-validation accuracy improving from 98.8% to 99.4% across 10,000 to 35,000 training instances. Both learning curves show the narrow and converging gap between training and validation scores indicating that the models aren't just memorizing training data but are also effectively learning generalizable patterns.

After initialization, models' comparison across diverse sets of sixteen supervised learning algorithms with rankings based on accuracy was done using PyCaret. The comparison results are shown in Section IV.

IV. Results

This section presents the experimental outcomes obtained from the supervised machine-learning pipeline applied to the CICIDS-2017 dataset. Results are reported for both the balanced binary classification task (Benign vs. Attack) and the SMOTE-enhanced multiclass classification task. All models were trained and evaluated using the PyCaret AutoML framework under same setup, training, and cross-validation conditions to ensure methodological fairness and reproducibility.

A. Evaluation Metrics

Although PyCaret automatically reports multiple evaluation metrics such as accuracy, precision, recall, AUC, Cohen's Kappa, and MCC, all of these metrics were not used as the primary decision criteria. Accuracy was selected as the primary performance metric because the datasets were balanced using sampling, and it helps to find the true detection of attacks from the balanced datasets. Training time was considered to ensure computational efficiency. The accuracy percent per attack was also calculated for each top model for all attacks with the help of confusion matrices obtained from multi classification task.

The selected evaluation metrics are defined as follows:

True Positive (TP): A true positive occurs when the model correctly identifies an instance belonging to the positive class. In network intrusion detection, this represents a successful detection of an actual attack.

True Negative (TN): A true negative occurs when the model correctly identifies an instance belonging to the negative class. This represents legitimate network traffic that is correctly classified as benign.

False Positive (FP): A false positive occurs when the model incorrectly classifies a negative instance as positive. IT represents benign attack being incorrectly classified as anomaly.

False Negative (FN): A false negative occurs when the model fails to identify a positive instance, incorrectly classifying it as negative. This represents an error in intrusion detection systems when anomaly attacks are not detected by model.

Accuracy: It is the most suitable metric for this experiment because we created balanced datasets for both binary and multiclassification. It is the ratio of correct predictions (true positives + true negatives) to the total predictions made which is defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

Precision: Precision measures the proportion of positive predictions that are actually correct. In intrusion detection, high precision means low false alarms. Precision is defined as:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

Recall: Recall measures the proportion of actual positive instances that are correctly identified by the model. In network security, high recall ensures that the system detects the maximum number of attacks, minimizing undetected intrusions. Recall is defined as:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3}$$

F1 Score: The F1 score is the harmonic mean of precision and recall, providing a balanced metric between both mea-

sures. It is particularly useful when both detecting attacks and avoiding false alarms are equally important. It is defined as:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

Or equivalently:

$$\text{F1 Score} = \frac{2 \times TP}{2 \times TP + FP + FN} \tag{5}$$

Computation Time (TT): It is the time it taken by model to train. It is a crucial metric in practical applications where computer efficiency and model reproducibility matter. TT is typically measured in second (s).

Per-Class Accuracy: For multiclass classification, per-class accuracy measures the proportion of instances of a specific attack type correctly identified. For attack type X, it is defined as:

$$\text{Per-Class Accuracy}_X = \frac{TP_X}{TP_X + FN_X} \tag{6}$$

This metric helps identify which models excel at detecting specific attack categories.

B. Binary Classification Results

The balanced binary dataset was used to evaluate models' ability to differentiate normal attack from anomalous attacks. Across the sixteen supervised algorithms examined, Catboost Classifier achieved the highest predictive performance, obtaining an accuracy of 99.24%. xgboost, lightgbm, extra trees classifier, and random forest followed closely behind catboost.

TABLE VI: Model Comparison Results for Binary Classification

Model	Accuracy	Recall	Precision	F1	Time (s)
CatBoost	0.9924	0.9919	0.9929	0.9924	7.83
XGBoost	0.9921	0.9922	0.9920	0.9921	0.16
LightGBM	0.9917	0.9907	0.9927	0.9917	0.33
Extra Trees	0.9914	0.9902	0.9927	0.9914	0.18
Random Forest	0.9906	0.9899	0.9914	0.9907	0.68
Decision Tree	0.9848	0.9876	0.9822	0.9849	0.13
Gradient Boost	0.9814	0.9835	0.9796	0.9815	2.46
K-Neighbors	0.9786	0.9845	0.9731	0.9788	0.72
AdaBoost	0.9621	0.9671	0.9578	0.9624	0.54
Logistic Reg.	0.9246	0.9618	0.8956	0.9275	1.14
SVM-Linear	0.9179	0.9401	0.9010	0.9197	0.03
QDA	0.9059	0.9765	0.8563	0.9124	0.03
Ridge	0.8948	0.9573	0.8516	0.9013	0.02
LDA	0.8905	0.9581	0.8447	0.8978	0.02
Naive Bayes	0.8645	0.9316	0.8257	0.8720	0.02
Dummy	0.5017	1.0000	0.5017	0.6682	0.02

Although accuracy was the primary ranking criteria, computational efficiency was also considered. So, the top models with accuracy above 99 percent were further ranked in aggregation with time taken as well as shown in Table VII. In this regard, Xgboost emerged as the highly accurate and fastest model with accuracy above 99 percent, requiring only 0.158 seconds of training time while still achieving 99.21% accuracy. This highlights xgboost as a better practical choice for environments with limited computational resources, whereas lightgbm offers superior classification strength at the cost of slightly higher computation.

TABLE VII: Quick Ranking of the Top Models in aggregration with Accuracy $\geq$ 0.99 and Training Time(sec)

Model	Accuracy	Training Time (sec)
XGBoost	0.9921	0.158
LightGBM	0.9917	0.325
Extra Trees	0.9914	0.178
CatBoost	0.9924	7.830
Random Forest	0.9906	0.682

C. Multiclass Classification Results

The multiclass classification experiment required distinguishing among nine attack categories in addition to benign traffic. After applying SMOTE to balance minority classes, PyCaret evaluated the same set of fifteen classifiers using identical cross-validation procedures. In this setting, LightGBM produced the best results, achieving 99.27% accuracy with the highest accuracy among all models tested.

TABLE VIII: Model Comparison Results for Multi Classification

Model	Accuracy	Recall	Precision	F1	Time (s)
LightGBM	0.9942	0.9942	0.9942	0.9942	2.25
Extra Trees	0.9940	0.9940	0.9940	0.9940	0.67
Random Forest	0.9935	0.9935	0.9935	0.9935	2.71
CatBoost	0.9926	0.9926	0.9926	0.9926	31.26
Decision Tree	0.9890	0.9890	0.9890	0.9890	0.40
Gradient Boost	0.9889	0.9889	0.9889	0.9889	50.81
K-Neighbors	0.9852	0.9852	0.9852	0.9851	0.42
QDA	0.9838	0.9838	0.9839	0.9838	0.08
Logistic Reg.	0.9541	0.9541	0.9554	0.9531	0.96
SVM-Linear	0.9422	0.9422	0.9441	0.9412	0.19
LDA	0.8323	0.8323	0.8475	0.8265	0.09
Ridge	0.7973	0.7973	0.8179	0.7937	0.07
Naive Bayes	0.7504	0.7504	0.7711	0.7334	0.06
AdaBoost	0.3416	0.3416	0.3663	0.2504	2.06
Dummy	0.1429	0.1429	0.0204	0.0357	0.08

Regarding computational efficiency, Extra Trees Classifier (et) was the fastest model among models with F1-Score above 99 percent, completing training in 0.670 seconds while reaching 99.40% accuracy. Although et performed slightly lower than lightgbm, it's lower training time makes it an ideal candidate for rapid retraining. After et, lightbgm, rf and catboost were ranked successively as shown in Table IX.

TABLE IX: Quick Ranking of the Top Models in aggregration with Accuracy $\geq$ 0.99 and Training Time(sec)

Model	Accuracy	Training Time (sec)
Extra Trees	0.9940	0.670
LightGBM	0.9942	2.248
Random Forest	0.9935	2.713
CatBoost	0.9926	31.263

Multiclassification can still be tricky as some models can have high accuracy but can still be biased more towards certain attacks. Some models can classify certain attacks more. So, it can be better if we can also classify the models which can accurately detect the specific types of attacks more and can cause less alarms for specific attack types. In this case, the

confusion matrix is perfect as they give detailed analysis of model prediction performance. Confusion matrices for best performing models are shown in Fig. 5 and the attack specific model performance table based on confusion matrixes is shown in Table X.

TABLE X: Attack Based Model Classification Based on Accuracy Rate

Attack Type	Extra Trees	LGBM	Random Forest	CatBoost
BENIGN	0.9829	0.9838	0.9808	0.9700
Bot	0.9988	0.9975	0.9979	0.9996
Brute Force	0.9975	0.9963	0.9979	0.9963
DDoS	0.9996	0.9996	0.9988	0.9996
DoS	0.9954	0.9954	0.9950	0.9958
Port Scan	0.9967	0.9971	0.9975	0.9975
Web Attack	0.9959	0.9958	0.9979	0.9963

The above Table X reveals the class-specific performance variations that could not be fully seen through metrics like accuracy and others. LGBM demonstrated the best performance in correctly classifying benign traffic (0.9838), CatBoost excelled at Bot detection with the highest accuracy of 0.9996, while Random Forest performed best for Brute Force (0.9979), Port Scan (0.9975), and Web Attack (0.9979) detection. For DDoS attacks, three models (Extra Trees, LGBM, and CatBoost) achieved equally high performance at 0.9996. This observation suggests that ensemble approaches combining specialized classifiers for different attack types may yield improved overall performance.

V. Comparison of Our Work with Similar Other Works in the Field

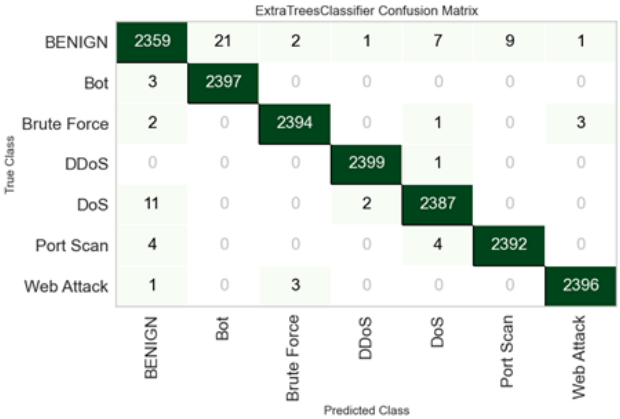
Zhaoyang Xu and Tunbo Liu evaluated the performance of MLP, CNN, LOF and OCSVM machine learning approaches on CICIDS 2017 dataset. Their models produced significantly higher performances on the testing set of the dataset, but our models perform even better and even have better time efficiency to run across most commercial computers [17].

TABLE XI: Performance on Overall Test Set from [17] for Binary Classification

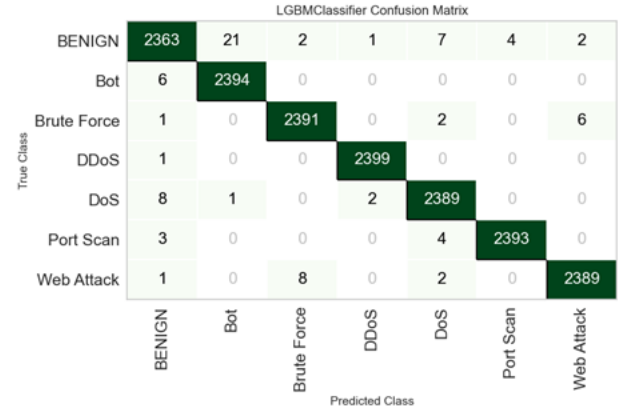
Model	Accuracy	Precision	Recall	F1-score
MLP	0.9775	0.9894	0.9036	0.9946
CNN	0.9650	0.9316	0.9010	0.9160
LOF	0.8046	0.9106	0.8341	0.8706
OCSVM	0.8356	0.6525	0.4784	0.5520

TABLE XII: Performance of Our Models for Binary Classification

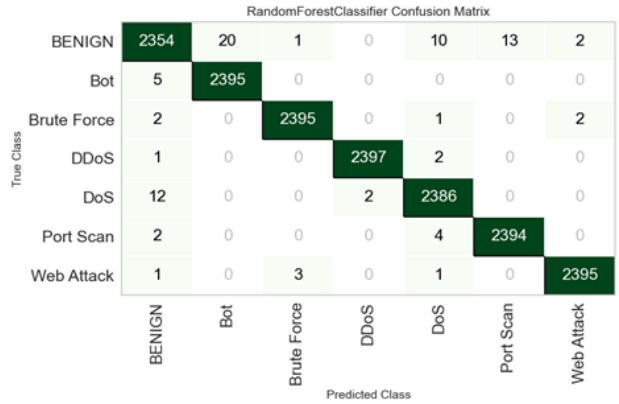
Model	Accuracy	Precision	Recall	F1-score
CatBoost	0.9924	0.9929	0.9919	0.9924
XGBoost	0.9921	0.9920	0.9922	0.9921
LightGBM	0.9917	0.9927	0.9907	0.9917
Extra Trees	0.9914	0.9927	0.9902	0.9914



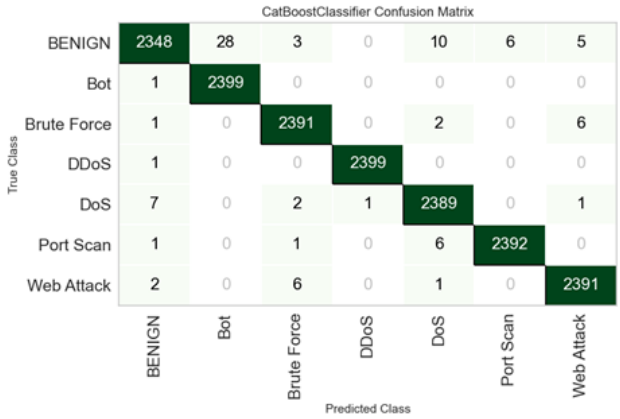
(a) Extra Trees



(b) LightGBM



(c) Random Forest



(d) CatBoostClassifier

Fig. 5: Confusion Matrix for Top 4 Models (Multiclass Classification)

TABLE XIII: Comparison Table of Models from [17] and Our Experiment for Individual Attack Type

Attack Type	Prior Work Best (From [17])	Our Best Model
DoS (aggregated)	CNN: 0.9978 (Hulk)	LGBM: 0.9954
BENIGN	MLP: 0.9974	LGBM: 0.9838
Port Scan	MLP: 0.9995	RF: 0.9975
DDoS	CNN: 0.9996	LGBM/ET/CatBoost: 0.9996
Brute Force	FTP-Patator: 0.9987	RF: 0.9979
Web Attack (aggregated)	Best: 0.2500 (SQL Injection, MLP)	ExtraTrees: 0.9983
Bot	LOF: 0.4680	CatBoost: 0.9996

Our approach demonstrates substantial improvements over prior work presented by Zhaoyang Xu and Tunbo Liu, particularly in detecting attack types that posed significant challenges for traditional machine learning and deep learning methods. The most dramatic improvement is in Web Attack detection, where our ExtraTrees model achieves an accuracy of 0.9983 compared to the prior work's best performance of 0.2500, representing an improvement of +0.7483. The earlier study struggled severely with web-based attacks, with XSS detection yielding accuracies between 0.0231 and 0.0385, and SQL Injection detection ranging from 0.0000 to 0.2500.

Similarly, Bot detection shows remarkable improvement, with our CatBoost model reaching 0.9996 accuracy compared to the prior work's best result of 0.4680 using LOF, an improvement of +0.5316. Beyond these dramatic gains in specific attack types, our approach achieves significantly more consistent performance across all categories, with accuracies ranging from 0.9700 to 0.9996 (spread of 0.0296) compared to prior work's extreme variance of 0.0000 to 1.0000. All seven of our attack categories achieve accuracy above 0.9500, whereas only seven of the fifteen categories in the prior study reached this threshold.

For attack types where prior work already demonstrated strong performance, such as DDoS (0.9996 vs 0.9996) and DoS (0.9954 vs 0.9978), our results remain competitive with negligible differences. The superior and consistent performance of our approach can be attributed to the strategic attack ingestion and class balancing, and the effectiveness of tree-based models in capturing complex network traffic patterns without requiring extensive computational resources.

Sugandh Seth et al. also presented a machine learning pipeline for intrusion detection including null values removal, outlier removal, undersampling, feature selection using random forest, dimensionality reduction for CICIDS 2018 dataset and comparing models like as Random Forests, Extra Trees, XGBoost, Histogram Gradient Boosting and kNN, and LightGBM and found out that every model compared above provided high predictive performance and faster model building time and low prediction latency which proves that our pipeline and the best models we found from the experiment on CICIDS 2017 dataset are also acceptable [18], [20].

Similarly, Dongzi Jin, Yiqin Lu et al. compared the lightgbm, xgboost and random forest models on CICIDS 2017 dataset and found out that lightgbm provided least model training time and prediction latency across different networks and followed by xgboost and random forest. This work is also the indication of how the best models we found if used individually can perform while pipeline reproduction and in real time environments [19].

VI. Future Considerations

This study focused exclusively on supervised learning for known attack categories. Future work should explore hybrid approaches combining supervised classification with unsupervised anomaly detection to identify attacks like Heartbleed and Infiltration that were not included for this experiment because of the very lower samples present in training data.
While training time was measured, comprehensive evaluation of inference latency in production environments with streaming network data and resource-constrained edge devices was not done, so it is necessary to find the best models in terms of real network environment as well.

Finally, model interpretability deserves attention as well. It was not done for this experiment because IPCA was used. IPCA combines the features to make its own features, which do not match real-time dataset features, due to which doing model interpretability becomes invaluable in our context of experiment. In the future, in place of IPCA, other techniques supporting model interpretability can be explored, as black box models are not sufficient to understand how models detect the attacks. After that, implementing explainability frameworks such as SHAP or LIME would help to find out what the most important features affecting the prediction of the model are.

VII. Discussion

Looking at the results, it's clear that tree-based ensemble models like XGBoost, LightGBM, CatBoost, and Extra Trees perform better than most other models, both for multiclass and binary intrusion detection. These models are simply better at picking up complex and nonlinear patterns in network traffic, which is something simpler models struggle with. This also explains why models like Naive Bayes and the Dummy Classifier performed poorly, since their assumptions don't really match how network data behaves in real life.

Another noticeable trend is that binary classification is easier than multiclass classification. Most models achieved higher and more consistent accuracy when the task was just to separate normal traffic from attacks. Once the problem shifts to identifying specific attack types, performance drops a bit, which makes sense given how similar some attack patterns can be.

There's also a clear trade-off between performance and training time. CatBoost achieved very strong results, but it took much longer to train, which could be a problem if the model needs frequent updates. On the other hand, models like Extra Trees and LightGBM offered a good balance, delivering high performance without excessive training costs.

The effectiveness of the preprocessing pipeline, particularly the combination of downsampling for binary classification and SMOTE for multiclass scenarios, contributed significantly to model performance. These techniques successfully addressed class imbalance which is one of the biggest problem in anomaly detection. Learning curves further validated the approach, showing minimal gaps between training and validation scores, indicating strong generalization capabilities and absence of overfitting.

VIII. Conclusion

This study presents a comprehensive evaluation of sixteen supervised learning algorithms for network intrusion detection using the CICIDS-2017 dataset, demonstrating how AutoML frameworks like PyCaret can streamline model comparison and selection for intrusion detection research. By combining systematic preprocessing techniques, including Incremental PCA, outlier removal, and balanced resampling strategies with PyCaret's automated evaluation capabilities, this research establishes a reproducible baseline methodology that enables informed model selection based on operational requirements and resource constraints.

The results demonstrate that tree-based ensemble methods achieve superior performance for both binary and multiclass intrusion detection tasks. XGBoost emerged as the optimal choice for binary classification (99.21% accuracy, 0.158s training time), while Extra Trees Classifier excelled in multiclass scenarios (99.40% accuracy, 0.670s training time). The confusion matrix analysis revealed the strengths of the attack-specific model: LGBM achieved the best benign traffic classification (0.9838), CatBoost excelled in Bot detection (0.9996), and Random Forest performed best for Brute Force (0.9979), Port Scan (0.9975) and Web Attack detection (0.9979).

In conclusion, this study contributes to intrusion detection research by building a computationally efficient machine learning pipeline and providing comprehensive guidance for model selection to detect network anomaly attacks.

Acknowledgements

This study was supported by The University of Southern Mississippi, Hattiesburg.

References

[1] N. Moustafa and J. Slay, "The significant features of the UNSW-NB15 and the KDD99 data sets for network intrusion detection systems," in *2015 IEEE Trustcom/BigDataSE/ISPA*, 2016, pp. 25–30.

[2] P. García-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Computers & Security*, vol. 28, no. 1–2, pp. 18–28, 2009.

[3] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[4] F. Hutter, L. Kotthoff, and J. Vanschoren, Eds., *Automated machine learning: Methods, systems, challenges*. Springer Nature, 2019.

[5] M. Ali, "PyCaret: An open source, low-code machine learning library in Python," 2020. [Online]. Available: https://www.pycaret.org.

[6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[7] S. Bagui and K. Li, "Resampling imbalanced data for network intrusion detection datasets," *Journal of Big Data*, vol. 8, no. 1, p. 6, 2021.

[8] I. T. Jolliffe and J. Cadima, "Principal component analysis: A review and recent developments," *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 374, no. 2065, p. 20150202, 2016.

[9] H. Liu, X. Wang, F. He, and Z. Zheng, "Automated network defense: A systematic review and analysis of AutoML paradigms for network intrusion detection," *Appl. Sci.*, vol. 15, no. 19, p. 10389, Sep. 2025.

[10] S. Ness, V. Eswarakrishnan, H. Sridharan, V. Shinde, N. V. P. Janapareddy, and V. Dhanawat, "Anomaly detection in network traffic using advanced machine learning techniques," *IEEE Access*, vol. 13, pp. 16133–16149, Jan. 2025.

[11] A. Dahal, P. Bajgai, and N. Rahimi, "Analysis of Zero Day Attack Detection Using MLP and XAI," in *Security and Management and Wireless Networks*, vol. 2254, Communications in Computer and Information Science, pp. 57–67, Jul. 2024.

[12] S. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. Chan, "KDD Cup 1999 data," UCI Machine Learning Repository, 1999.

[13] Z. Yang, X. Liu, T. Li, D. Wu, J. Wang, Y. Zhao, and H. Han, "A systematic literature review of methods and datasets for anomaly-based network intrusion detection," *Computers & Security*, vol. 116, p. 102675, May 2022, doi: 10.1016/j.cose.2022.102675.

[14] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms," in *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, Chicago, IL, USA, 2013, pp. 847–855. doi: 10.1145/2487575.2487629.

[15] E. LeDell and S. Poirier, "H2O AutoML: Scalable automatic machine learning," in *Proceedings of the AutoML Workshop at ICML 2020*, 2020.

[16] M. A. Zoller and M. F. Huber, "Benchmark and survey of automated machine learning frameworks," arXiv preprint arXiv:1904.12054, 2019.

[17] Z. Xu and Y. Liu, "Robust anomaly detection in network traffic: Evaluating machine learning models on CICIDS2017," arXiv preprint arXiv:2506.19877, 2025.

[18] S. Seth, G. Singh, and K. Kaur Chahal, "A novel time efficient learning-based approach for smart intrusion detection system," *Journal of Big Data*, vol. 8, p. 111, 2021.

[19] D. Jin, Y. Lu, J. Qin, Z. Cheng, and Z. Mao, "SwiftIDS: Real-time intrusion detection system based on LightGBM and parallel intrusion detection mechanism," *Computers & Security*, vol. 97, p. 101984, 2020.

[20] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2018, pp. 108–116.